

Pixel Battle — Visual Polish (Sub-project E) — Implementation Plan

Visual Polish (Sub-project E) Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development. Steps use checkbox (– []) syntax for tracking. User has pre-approved everything; skip approval gates.

Goal: Wrap the existing scripted-fight renderer in a KOF-style HUD + impact-feedback layer (top health bars, beefier hit sparks, KO splash with slow-motion + zoom).

Architecture: Three new pygame draw modules sit on top of the existing world layer in `_render_fight`. Engine, characters, skills, timeline driver — all untouched. The KO controller modifies the per-tick `dt_ms` passed into the engine to produce slow-motion without altering engine internals.

Tech Stack: Native pygame (`pygame.draw`, `pygame.Surface`, `SRCALPHA`), numpy for impact-FX particle math. No new pip dependency.

Spec: `docs/superpowers/specs/2026-05-25-pixel-battle-visual-polish-design.md`.

File Structure

Create: – `pixel_battle/rl/hud.py` — HUD class (top health bars + name plates + match timer) – `pixel_battle/rl/impact_fx.py` — ImpactFX registry (big sparks, screen flash, floating text) – `pixel_battle/rl/ko_sequence.py` —

KO state machine (impact / slow-mo / hold) –
pixel_battle/tests/test_hud.py – pixel_battle/tests/test_impact_fx.py –
pixel_battle/tests/test_ko_sequence.py

Modify: – pixel_battle/rl/play.py::_render_fight — instantiate the three new modules; route engine events; multiply the engine's dt_ms by the KO controller's dt_scale; composite HUD + impact FX on top of the world surface.

Pre-work for every task

Before each task, the implementer must read: – pixel_battle/rl/play.py — _render_fight is the integration point. Understand the existing tick loop, camera-zoom variable, event consumption pattern, and recorder.write call. – pixel_battle/rl/stick_renderer.py — existing spawn_impact_burst, spawn_landing_dust, spawn_flash_puff patterns. New modules follow the same idiom. – pixel_battle/engine/battle.py — Event / EventType shape (the extra: dict field carries hit metadata). – pixel_battle/engine/character.py — hp, hp_max, pos_x, id fields used by HUD. – pixel_battle/data/characters.json — each character's brand color (used in HUD bars).

The implementer is empowered to adjust pixel positions, colors, and timings if the spec values look wrong against the actual world surface dimensions and font sizes. Match the spec's INTENT, not its literal numbers.

Task 1: HUD overlay (hud.py)

KOF-style top strip: two health bars + name plates + center timer.

Files: – Create: pixel_battle/rl/hud.py – Test: pixel_battle/tests/test_hud.py

☐ **Step 1: Write the failing test**

```
# pixel_battle/tests/test_hud.py
import os
os.environ.setdefault("SDL_VIDEODRIVER", "dummy")
import pygame
pygame.init()
pygame.font.init()
pygame.display.set_mode((1, 1))
```

```

from pixel_battle.rl.hud import HUD
from pixel_battle.engine.character import Character
from pixel_battle.engine.battle import Battle
from pixel_battle.engine.rng import BattleRNG

def _battle():
    left = Character.load("garen")
    right = Character.load("lux")
    return Battle(left, right, rng=BattleRNG(seed=0))

def test_hud_draws_left_and_right_bars():
    b = _battle()
    surf = pygame.Surface((480, 854))
    surf.fill((0, 0, 0))
    HUD().draw(surf, b, elapsed_ms=0)
    # Top strip should now have non-zero pixels on both halves
    arr = pygame.surfarray.array3d(surf)
    assert arr[:240, :70].any(), "left half of HUD strip is empty"
    assert arr[240:, :70].any(), "right half of HUD strip is empty"

def test_hud_health_bar_drains_with_hp_loss():
    b = _battle()
    surf_full = pygame.Surface((480, 854))
    HUD().draw(surf_full, b, elapsed_ms=0)
    full_left_pixels = (pygame.surfarray.array3d(surf_full)[:240, :70]
                        > 0).sum()
    # Smash left's HP to 1, settle the lerp by ticking the HUD several times
    b.left.hp = 1
    hud = HUD()
    surf_drained = pygame.Surface((480, 854))
    for _ in range(60): # let smoothed-drain catch up over ~1 s of 60 fps frames
        surf_drained.fill((0, 0, 0))
        hud.draw(surf_drained, b, elapsed_ms=1000)
    drained_left_pixels = (pygame.surfarray.array3d(surf_drained)
                          [:240, :70] > 0).sum()
    assert drained_left_pixels < full_left_pixels, (
        f"left bar didn't shrink: full={full_left_pixels} drained={drained_left_pixels}")

def test_hud_timer_renders():
    b = _battle()
    surf = pygame.Surface((480, 854))
    surf.fill((0, 0, 0))
    HUD().draw(surf, b, elapsed_ms=12500)
    arr = pygame.surfarray.array3d(surf)

```

```
# The center 60px column above the world line should have rendered text
```

```
assert arr[210:270, 40:65].any()
```

☐ Step 2: Run test to verify it fails

Run: `python -m pytest pixel_battle/tests/test_hud.py -v` Expected: FAIL with `ModuleNotFoundError: No module named 'pixel_battle.rl.hud'`.

☐ Step 3: Implement HUD

```
# pixel_battle/rl/hud.py
```

```
"""KOF-style top HUD: two health bars + name plates + match timer.
```

```
Stateless except for the smoothed-bar lerp state, which is per-HUD-instance.
```

```
A new HUD is constructed per render in `_render_fight`."""
```

```
from __future__ import annotations
```

```
import json
```

```
import pygame
```

```
from pixel_battle.engine.character import DATA_PATH
```

```
HUD_HEIGHT = 70 # px from top reserved for HUD
```

```
BAR_WIDTH = 180
```

```
BAR_HEIGHT = 16
```

```
BAR_Y = 28
```

```
NAME_FONT_SIZE = 14
```

```
TIMER_FONT_SIZE = 14
```

```
BAR_DRAIN_LERP_RATE = 0.12 # per-frame towards target HP fraction
```

```
BAR_BG = (40, 40, 50)
```

```
BAR_BORDER = (220, 220, 220)
```

```
NAME_COLOR = (230, 230, 230)
```

```
NAME_SHADOW = (10, 10, 14)
```

```
TIMER_COLOR = (180, 180, 180)
```

```
def _load_char_color(char_id: str) -> tuple:
```

```
"""Read the character's brand color from data, default to white."""
```

```
with open(DATA_PATH, encoding="utf-8") as f:
```

```
    data = json.load(f)
```

```
    rgb = data.get(char_id, {}).get("brand_color")
```

```
    if not rgb:
```

```
        return (220, 220, 220)
```

```
    return tuple(rgb[:3])
```

```
class HUD:
```

```
"""KOF-style top HUD. Construct once per render; call `draw` each frame."""
```

```

def __init__(self):
    self._left_lerp = 1.0
    self._right_lerp = 1.0
    self._name_font = pygame.font.SysFont(None, NAME_FONT_SIZE *
2)
    self._timer_font = pygame.font.SysFont(None, TIMER_FONT_SIZE *
2)
    self._color_cache: dict = {}

def _color(self, char_id: str):
    if char_id not in self._color_cache:
        self._color_cache[char_id] = _load_char_color(char_id)
    return self._color_cache[char_id]

def draw(self, surf: pygame.Surface, battle, elapsed_ms: int) ->
None:
    W = surf.get_width()
    left_frac = max(0.0, battle.left.hp / max(1,
battle.left.hp_max))
    right_frac = max(0.0, battle.right.hp / max(1,
battle.right.hp_max))
    self._left_lerp += (left_frac - self._left_lerp) *
BAR_DRAIN_LERP_RATE
    self._right_lerp += (right_frac - self._right_lerp) *
BAR_DRAIN_LERP_RATE
    # left bar (drains right-to-left toward center)
    self._draw_bar(surf, x_right=W // 2 - 30, width=BAR_WIDTH,
frac=self._left_lerp,
                    color=self._color(battle.left.id),
                    direction=-1)
    # right bar (drains left-to-right toward center)
    self._draw_bar(surf, x_left=W // 2 + 30, width=BAR_WIDTH,
frac=self._right_lerp,
                    color=self._color(battle.right.id),
                    direction=+1)
    # name plates
    self._draw_name(surf, battle.left.id, x=W // 2 - 30 -
BAR_WIDTH, align="left")
    self._draw_name(surf, battle.right.id, x=W // 2 + 30 +
BAR_WIDTH, align="right")
    # center timer
    self._draw_timer(surf, elapsed_ms, x=W // 2, y=BAR_Y +
BAR_HEIGHT + 2)

def _draw_bar(self, surf, frac, color, width, direction,
x_right=None, x_left=None):
    if x_right is not None:
        x = x_right - width
    else:
        x = x_left
    pygame.draw.rect(surf, BAR_BG, (x, BAR_Y, width, BAR_HEIGHT))
    fill_w = max(0, int(width * frac))
    if direction == -1:
        fill_x = x + (width - fill_w)

```

```

else:
    fill_x = x
    pygame.draw.rect(surf, color, (fill_x, BAR_Y, fill_w,
    BAR_HEIGHT))
    pygame.draw.rect(surf, BAR_BORDER, (x, BAR_Y, width,
    BAR_HEIGHT), width=1)

def _draw_name(self, surf, name, x, align):
    text = name.replace("_", " ").upper()
    shadow = self._name_font.render(text, True, NAME_SHADOW)
    label = self._name_font.render(text, True, NAME_COLOR)
    if align == "left":
        x_pos = x
    else:
        x_pos = x - label.get_width()
    surf.blit(shadow, (x_pos + 1, BAR_Y + 1))
    surf.blit(label, (x_pos, BAR_Y))

def _draw_timer(self, surf, elapsed_ms, x, y):
    seconds = max(0, elapsed_ms // 1000)
    text = f"{seconds // 60:02d}:{seconds % 60:02d}"
    label = self._timer_font.render(text, True, TIMER_COLOR)
    surf.blit(label, (x - label.get_width() // 2, y))

```

If `characters.json` does not yet have a `brand_color` field per character, the implementer may add one with a sensible default (per character — Garen blue, Lux yellow, etc) — this counts as a data-only change, not an engine change.

☐ Step 4: Run test to verify it passes

Run: `python -m pytest pixel_battle/tests/test_hud.py -v` Expected: PASS — 3/3.

If `test_hud_health_bar_drains_with_hp_loss` fails because the lerp converges too slowly: increase `BAR_DRAIN_LERP_RATE`. If the timer-pixel-region assertion fails because the timer renders at a different position: adjust the test's expected pixel rect to where the timer actually lands. Match the spec's INTENT, not its literal numbers.

☐ Step 5: Commit

```

git add pixel_battle/rl/hud.py pixel_battle/tests/test_hud.py
    pixel_battle/data/characters.json
git commit -m "feat(pixel-battle/rl): KOF-style top HUD — health bars,
    name plates, timer"

```

Task 2: Impact FX (impact_fx.py)

Owns active impact effects, spawn helpers, per-frame update + draw.
Integrates with `_render_fight`'s existing event loop.

Files: – Create: `pixel_battle/rl/impact_fx.py` – Test:
`pixel_battle/tests/test_impact_fx.py`

☐ Step 1: Write the failing test

```
# pixel_battle/tests/test_impact_fx.py
import os
os.environ.setdefault("SDL_VIDEODRIVER", "dummy")
import pygame
pygame.init()
pygame.font.init()
pygame.display.set_mode((1, 1))

from pixel_battle.rl.impact_fx import ImpactFX

def test_spark_burst_marks_pixels():
    fx = ImpactFX()
    surf = pygame.Surface((480, 854))
    surf.fill((0, 0, 0))
    fx.spawn_hit_spark(x=240, y=400, damage=5, color=(255, 220, 100))
    fx.update_and_draw(surf, dt_ms=16)
    arr = pygame.surfarray.array3d(surf)
    near = arr[210:270, 380:420]
    assert near.any(), "spark should mark pixels near the hit point"

def test_spark_count_scales_with_damage():
    fx_small = ImpactFX()
    fx_big = ImpactFX()
    fx_small.spawn_hit_spark(x=240, y=400, damage=1, color=(255, 255, 255))
    fx_big.spawn_hit_spark(x=240, y=400, damage=20, color=(255, 255, 255))
    assert len(fx_big._active) > len(fx_small._active), (
        f"damage=20 should spawn more particles than damage=1: "
        f"small={len(fx_small._active)} big={len(fx_big._active)}")

def test_screen_flash_overlays_color():
    fx = ImpactFX()
    surf = pygame.Surface((480, 854))
    surf.fill((0, 0, 0))
    fx.flash_screen(color=(255, 0, 0), alpha=180)
    fx.update_and_draw(surf, dt_ms=16)
    arr = pygame.surfarray.array3d(surf)
```

```

# Some red should have made it through onto the whole frame
red_pixels = (arr[:, :, 0] > 100).sum()
assert red_pixels > 1000, f"red flash didn't paint enough pixels:
    {red_pixels}"

def test_floating_text_rises_and_expires():
    fx = ImpactFX()
    fx.spawn_floating_text(x=240, y=400, text="HIT!", color=(255, 255,
        0))
    assert len(fx._texts) == 1
    # advance 500 ms - past the default lifetime
    for _ in range(35):
        surf = pygame.Surface((480, 854))
        fx.update_and_draw(surf, dt_ms=16)
    assert len(fx._texts) == 0, "floating text should have expired"

```

☐ Step 2: Run test to verify it fails

Run: `python -m pytest pixel_battle/tests/test_impact_fx.py -v` Expected:
 FAIL with `ModuleNotFoundError: No module named`
 'pixel_battle.rl.impact_fx'.

☐ Step 3: Implement ImpactFX

```

# pixel_battle/rl/impact_fx.py
"""Renderer-side impact effects: big sparks, screen flash, floating
    text.

```

```

Stateless per spawn; `update_and_draw` ticks all active effects and
    renders
    them onto the supplied world surface. Spawning is decoupled from
    drawing
    so `_render_fight` can route engine events to the spawn helpers and
    the
    per-frame draw step composites everything."""

```

```

from __future__ import annotations
import math
import random
from dataclasses import dataclass
from typing import List, Tuple

```

```

import pygame

```

```

TEXT_FONT_SIZE = 22
TEXT_LIFETIME_MS = 400
TEXT_RISE_PX = 30
FLASH_DECAY_PER_FRAME = 18

```

```

@dataclass
class _Spark:

```



```

x: float
y: float
vx: float
vy: float
life_ms: int
color: Tuple[int, int, int]

```

```
@dataclass
```

```
class _FloatingText:
```

```

    x: int
    y: int
    text: str
    color: Tuple[int, int, int]
    age_ms: int = 0

```

```
class ImpactFX:
```

```

    def __init__(self):
        self._active: List[_Spark] = []
        self._texts: List[_FloatingText] = []
        self._flash_color = (255, 255, 255)
        self._flash_alpha = 0
        self._text_font = pygame.font.SysFont(None, TEXT_FONT_SIZE * 2)

```

```

    def spawn_hit_spark(self, x: int, y: int, damage: int,
                        color: Tuple[int, int, int]) -> None:

```

```

        n = max(6, min(28, 6 + damage * 2))
        for _ in range(n):
            ang = random.uniform(0, math.tau)
            speed = random.uniform(2.5, 5.5)
            life = random.randint(140, 280)
            self._active.append(_Spark(
                x=x, y=y,
                vx=math.cos(ang) * speed, vy=math.sin(ang) * speed,
                life_ms=life, color=color))

```

```

    def flash_screen(self, color: Tuple[int, int, int], alpha: int = 200) -> None:

```

```

        self._flash_color = color
        self._flash_alpha = max(self._flash_alpha, alpha)

```

```

    def spawn_floating_text(self, x: int, y: int, text: str,
                           color: Tuple[int, int, int]) -> None:

```

```

        self._texts.append(_FloatingText(x=x, y=y, text=text,
                                          color=color))

```

```

    def update_and_draw(self, surf: pygame.Surface, dt_ms: int) -> None:

```

```

        # update sparks
        alive: List[_Spark] = []

```

```

for s in self._active:
    s.x += s.vx
    s.y += s.vy
    s.vy += 0.25 # gravity-like droop
    s.life_ms -= dt_ms
    if s.life_ms > 0:
        alive.append(s)
self._active = alive
# draw sparks
for s in self._active:
    pygame.draw.line(surf, s.color,
                     (int(s.x), int(s.y)),
                     (int(s.x - s.vx), int(s.y - s.vy)),
                     width=2)

# texts
alive_texts: List[FloatingText] = []
for t in self._texts:
    t.age_ms += dt_ms
    if t.age_ms < TEXT_LIFETIME_MS:
        alive_texts.append(t)
        frac = t.age_ms / TEXT_LIFETIME_MS
        y_offset = int(TEXT_RISE_PX * frac)
        fade = max(0, 255 - int(255 * frac))
        rendered = self._text_font.render(t.text, True,
        t.color)
        rendered.set_alpha(fade)
        surf.blit(rendered, (t.x - rendered.get_width() // 2,
                             t.y - y_offset -
                             rendered.get_height()))
self._texts = alive_texts
# screen flash
if self._flash_alpha > 0:
    overlay = pygame.Surface(surf.get_size(),
    flags=pygame.SRCALPHA)
    overlay.fill((*self._flash_color, self._flash_alpha))
    surf.blit(overlay, (0, 0))
    self._flash_alpha = max(0, self._flash_alpha -
    FLASH_DECAY_PER_FRAME)

```

☐ Step 4: Run test to verify it passes

Run: `python -m pytest pixel_battle/tests/test_impact_fx.py -v` Expected: PASS — 4/4. If a pixel-region assertion fails because the spark velocities sent the particles outside the test's pixel rect, widen the rect to where the actual sparks landed — match INTENT.

☐ Step 5: Commit

```

git add pixel_battle/rl/impact_fx.py
      pixel_battle/tests/test_impact_fx.py
git commit -m "feat(pixel-battle/rl): impact FX — big sparks, screen
      flash, floating text"

```

Task 3: KO sequence (ko_sequence.py)

State machine that takes over the last seconds of a fight: impact flash → slow-mo → hold + zoom.

Files: – Create: pixel_battle/rl/ko_sequence.py – Test: pixel_battle/tests/test_ko_sequence.py

☐ Step 1: Write the failing test

```
# pixel_battle/tests/test_ko_sequence.py
import os
os.environ.setdefault("SDL_VIDEODRIVER", "dummy")
import pygame
pygame.init()
pygame.display.set_mode((1, 1))

from pixel_battle.rl.ko_sequence import KOSequence

def test_starts_inactive_returns_normal_dt():
    seq = KOSequence()
    out = seq.tick(ko_active=False, ko_loser_x=240, dt_ms=16)
    assert out.dt_scale == 1.0
    assert out.zoom == 1.0
    assert out.splash_alpha == 0

def test_impact_state_flashes_and_spawns_splash():
    seq = KOSequence()
    out = seq.tick(ko_active=True, ko_loser_x=240, dt_ms=16)
    assert out.spawn_flash, "first KO tick should request a screen flash"
    assert out.spawn_splash, "first KO tick should request K.O. text"

def test_slowmo_returns_reduced_dt_scale():
    seq = KOSequence()
    # consume impact (0-200 ms)
    for _ in range(13):
        seq.tick(ko_active=True, ko_loser_x=240, dt_ms=16)
    out = seq.tick(ko_active=True, ko_loser_x=240, dt_ms=16)
    assert out.dt_scale < 0.5, (
        f"slow-mo dt_scale should be small, got {out.dt_scale}")
    assert out.zoom > 1.0, f"camera should be zooming, got {out.zoom}"

def test_hold_state_freezes_engine():
```

```

seq = K0Sequence()
# run through impact + slow-mo (~1.2 s total)
for _ in range(75):
    seq.tick(ko_active=True, ko_loser_x=240, dt_ms=16)
out = seq.tick(ko_active=True, ko_loser_x=240, dt_ms=16)
assert out.dt_scale == 0.0, "hold state should fully freeze the
    engine"
assert out.splash_alpha == 255, "K.O. splash should hold full
    opacity"

```

☐ Step 2: Run test to verify it fails

Run: `python -m pytest pixel_battle/tests/test_ko_sequence.py -v`

Expected: FAIL with `ModuleNotFoundError: No module named`

`'pixel_battle.rl.ko_sequence'.`

☐ Step 3: Implement K0Sequence

```

# pixel_battle/rl/ko_sequence.py
"""K0 sequence controller – impact flash → slow-motion → hold + zoom.

```

```

Per render. `tick(ko_active, ko_loser_x, dt_ms)` returns a
    `TickResult`
describing how `_render_fight` should drive this frame: how much to
    scale
the engine's `dt_ms`, the camera zoom, the splash alpha, and one-shot
    spawn signals."""

```

```

from __future__ import annotations
from dataclasses import dataclass

```

```

IMPACT_MS = 200
SLOWMO_MS = 1000
HOLD_MS = 1500
SLOWMO_DT_SCALE = 1.0 / 3.0
MAX_ZOOM = 1.6

```

```

@dataclass
class TickResult:
    dt_scale: float
    zoom: float
    zoom_focus_x: int
    splash_alpha: int
    spawn_flash: bool = False
    spawn_splash: bool = False

```

```

class K0Sequence:
    def __init__(self):
        self._t_ms = 0
        self._active = False
        self._spawned_flash = False

```

```

self._spawned_splash = False
self._loser_x = 240

def tick(self, ko_active: bool, ko_loser_x: int, dt_ms: int) ->
TickResult:
    if not ko_active:
        return TickResult(dt_scale=1.0, zoom=1.0,
            zoom_focus_x=240,
                           splash_alpha=0)
    if not self._active:
        self._active = True
        self._loser_x = ko_loser_x
        self._t_ms = 0
        self._spawned_flash = False
        self._spawned_splash = False

    result = TickResult(dt_scale=1.0, zoom=1.0,
        zoom_focus_x=self._loser_x,
                        splash_alpha=0)
    if self._t_ms < IMPACT_MS:
        # IMPACT
        if not self._spawned_flash:
            result.spawn_flash = True
            self._spawned_flash = True
        if not self._spawned_splash:
            result.spawn_splash = True
            self._spawned_splash = True
        frac = self._t_ms / IMPACT_MS
        result.dt_scale = SLOWMO_DT_SCALE
        result.zoom = 1.0 + (MAX_ZOOM - 1.0) * frac * 0.5
        result.splash_alpha = int(255 * frac)
    elif self._t_ms < IMPACT_MS + SLOWMO_MS:
        # SLOW-MO
        frac = (self._t_ms - IMPACT_MS) / SLOWMO_MS
        result.dt_scale = SLOWMO_DT_SCALE
        result.zoom = 1.0 + (MAX_ZOOM - 1.0) * (0.5 + 0.5 * frac)
        result.splash_alpha = 255
    else:
        # HOLD
        result.dt_scale = 0.0
        result.zoom = MAX_ZOOM
        result.splash_alpha = 255
    self._t_ms += dt_ms
    return result

```

- ☐ Step 4: Run test to verify it passes

Run: `python -m pytest pixel_battle/tests/test_ko_sequence.py -v`
 Expected: PASS — 4/4.

- ☐ Step 5: Commit

```
git add pixel_battle/rl/ko_sequence.py
      pixel_battle/tests/test_ko_sequence.py
git commit -m "feat(pixel-battle/rl): KO sequence - impact flash,
      slow-mo, zoom hold"
```

Task 4: Integration in `_render_fight` + validation render

Wire the three new modules into the renderer. Then render all 5 timeline scripts and confirm the result.

Files: – Modify: `pixel_battle/rl/play.py::_render_fight`

☐ Step 1: Read `_render_fight` carefully

Open `pixel_battle/rl/play.py`. Locate `_render_fight`. Understand: – The per-tick loop (where `env.battle.tick_ms(dt)` and `env.step(...)` are called). – The current camera-zoom state (variable name and how it's used by the world-surface composition). – The event consumption pattern — which events are already handled, what `event.extra` carries. – The recorder write call.

☐ Step 2: Inject the three modules

At the top of `_render_fight`, after env construction:

```
from pixel_battle.rl.hud import HUD
from pixel_battle.rl.impact_fx import ImpactFX
from pixel_battle.rl.ko_sequence import KOSequence
from pixel_battle.engine.battle import EventType, BattleState
```

```
hud = HUD()
fx = ImpactFX()
ko_seq = KOSequence()
events_consumed = 0      # index of next unconsumed event in
                        env.battle.events
```

☐ Step 3: Per-tick: route events to FX, then derive KO state

Inside the main loop, BEFORE drawing the world:

```
# Drain new engine events into the FX layer
while events_consumed < len(env.battle.events):
    ev = env.battle.events[events_consumed]
    events_consumed += 1
    if ev.type == EventType.HIT or ev.type == EventType.CRIT:
        # Find the target's screen position; use battle.right if
        ev.target=='right', etc.
```

```

target = env.battle.left if ev.target == env.battle.left.id
else env.battle.right
# Use the character's brand color for the spark (read via
HUD's helper or direct)
col = hud._color(target.id)
fx.spawn_hit_spark(x=int(target.pos_x), y=int(target.pos_y -
20),
                    damage=ev.amount, color=col)
if ev.type == EventType.CRIT or ev.extra.get("crit"):
    fx.flash_screen(color=(255, 240, 180), alpha=160)
    fx.spawn_floating_text(x=int(target.pos_x),
y=int(target.pos_y - 40),
                        text="CRIT!", color=(255, 220, 60))
else:
    fx.spawn_floating_text(x=int(target.pos_x),
y=int(target.pos_y - 40),
                        text="HIT!", color=(255, 240, 200))
elif ev.type == EventType.ULTIMATE_START:
    fx.flash_screen(color=(255, 60, 60), alpha=200)

# KO state driver
ko_active = env.battle.state == BattleState.KO
ko_loser_x = (int(env.battle.left.pos_x)
              if env.battle.left.hp <= 0 else
              int(env.battle.right.pos_x))
ko_result = ko_seq.tick(ko_active=ko_active, ko_loser_x=ko_loser_x,
dt_ms=16)
if ko_result.spawn_flash:
    fx.flash_screen(color=(255, 255, 255), alpha=255)
if ko_result.spawn_splash:
    fx.spawn_floating_text(x=240, y=420, text="K.O.!", color=(255, 80,
80))

# Scale the engine's tick by the KO controller
effective_dt = max(0, int(16 * ko_result.dt_scale))
# Replace existing `env.battle.tick_ms(16)` etc with the scaled value:
# env.battle.tick_ms(effective_dt)
# (find the existing tick call and replace its dt argument)

```

The implementer adapts the exact integration point (variable names, where in the loop the existing dt is passed) to whatever play.py actually does.

☐ Step 4: Per-tick: composite HUD + FX on top of the world surface

After the world surface is composed (existing code), BEFORE
recorder.write(...):

```

hud.draw(world, env.battle, elapsed_ms=env.battle.elapsed_ms)
fx.update_and_draw(world, dt_ms=16)
# If ko_result.zoom != 1.0, the camera should already be applying the
zoom via

```

```
# the existing CAM_ZOOM mechanism – pass ko_result.zoom into wherever
    play.py
# currently sets the per-frame zoom.
```

☐ Step 5: Run full suite — confirm nothing regresses

Run: `python -m pytest pixel_battle/tests/ -q` Expected: 423 + 11 new (3 HUD + 4 impact + 4 KO) = 434 passed.

If any pre-existing test regresses (e.g. `test_play_richness.py` asserts something specific about the world surface that the HUD now covers), update that test's expected pixel region to a part of the frame NOT occupied by the HUD strip. This is acceptable adaptation: the visual safety test on character poses (`test_all_poses_keep_feet_planted_and_in_frame`) MUST stay green; renderer richness tests CAN adapt to the new HUD strip.

☐ Step 6: Validation — render all 5 timeline scripts

```
for n in 01_lux_kite_garen 02_garen_rush_lux 03_glass_slow_brick
    04_yasuo_duel_ashe 05_lux_barrier_yasuo; do
    python -m pixel_battle.rl.play_scripted
        pixel_battle/data/scripts/${n}.yaml
done
```

Expected: – 5 mp4s in `pixel_battle/output/scripted/`. – File sizes in the same order of magnitude as before ($\approx$ 300KB–800KB). – No render crashes.

☐ Step 7: Commit

```
git add pixel_battle/rl/play.py
git commit -m "feat(pixel-battle/rl): integrate HUD + impact FX + KO
    sequence into _render_fight"
```